Université de Montpellier
Faculté des Sciences — Master 1 Informatique (Parcours ICO)
UE HAI704I : Architectures Logicielles Distribuées

Rapport de TP 1

Java RMI : Cabinet Vétérinaire Distribué

Auteur : Léonard ADAMO

Enseignant : Bachar RIMA

Année Universitaire : 2026 – 2027

Dépôt GitHub : https://github.com/Juste-Leo2/HAI704I-TP1

30 septembre 2026

Table des matières

1 Introduction 1

2 Étape A0 : Préparation, compilation et exécution 1

3 Étape A1 : Communication de base (Premier objet distant) 1

4 Étape A2 : Type d'espèce transmis par valeur 2

5 Étape A3 : Dossier de suivi médical partagé 2

6 Étape A4 : Le cabinet vétérinaire (Point d'entrée unique) 3

7 Étape A5 : Création d'un patient côté serveur 3

8 Étape A6 : Système d'alertes par callbacks (Abonnement) 4

9 Étape A7 : Ce qui traverse le réseau (Expérience de sérialisation) 4

10 Étape A8 : Application CLI interactive et architecture client 5

11 Partie C (Bonus) : Interface graphique Swing 6

12 Résultats et discussions 7

13 Usage des outils d'intelligence artificielle générative 7

14 Références 8

1 Introduction

L'objectif de ce TP est de concevoir et réaliser un système distribué de cabinet vétérinaire permettant la gestion de patients animaux (consultation, suivi médical, création). L'application est développée en Java RMI natif (JDK 21), avec la contrainte imposée de n'utiliser aucun framework externe.

L'architecture repose sur trois modules distincts : `common` (contrats d'interfaces et types partagés), `server` (implémentations et registre RMI) et `client` (interfaces utilisateur et logique applicative).

La progression du travail s'articule des étapes A0 à A8, prolongée par une interface graphique en bonus (Partie C) : préparation et validation de l'environnement Java, première communication distante avec l'animal, création du type espèce transmis par valeur, partage du dossier de suivi médical, point d'entrée unique avec le cabinet vétérinaire, ajout de patients depuis le client, système d'alertes par observateurs et tests de robustesse, étude de la sérialisation sur le réseau, puis réalisation d'une interface utilisateur console (CLI) et d'une vue graphique native (Swing). L'ensemble de ces étapes a été versionné pas à pas de manière transparente sur le dépôt GitHub.

2 Étape A0 : Préparation, compilation et exécution

Pour garantir une base saine et ne pas polluer l'arborescence des sources avec les fichiers compilés `.class`, un dossier `bin/` est créé dans chaque module (`common/bin`, `server/bin`, `client/bin`). Ces dossiers sont ignorés par git via le `.gitignore`.

La compilation s'effectue dans l'ordre strict des dépendances :

```
javac -d common/bin common/src/cabinet/*.java
javac -cp common/bin -d server/bin server/src/cabinet/*.java
javac -cp common/bin -d client/bin client/src/cabinet/*.java
```

Pour l'exécution, le serveur initialise le registre RMI sur le port 1099, puis les clients sont lancés dans des terminaux distincts :

```
# Linux / macOS (separateur :)
java -cp "server/bin:common/bin" cabinet.ServerMain
java -cp "client/bin:common/bin" cabinet.ClientMain

# Windows (separateur ;)
java -cp "server/bin;common/bin" cabinet.ServerMain
java -cp "client/bin;common/bin" cabinet.ClientMain
```

Le point de contrôle initial valide que le client ne dépend d'aucune classe d'implémentation du serveur, mais uniquement du module `common`.

3 Étape A1 : Communication de base (Premier objet distant)

La première étape met en place une communication distante minimale avec un objet `Animal` :

- Dans `common` : définition de l'interface distante `Animal` étendant `java.rmi.Remote`.
- Dans `server` : écriture du servant `AnimalImpl` étendant `UnicastRemoteObject`, puis publication dans le registre RMI via `registry.rebind("Animal", ...)`.
- Dans `client` : récupération de la référence par `registry.lookup("Animal")` et appel de méthodes.

L'appel à `Proxy.isProxyClass(stub.getClass())` renvoie `true` et le nom de la classe est de la forme `jdk.proxy1.$Proxy0`. Cela confirme que le client manipule bien un proxy dynamique (le stub) généré par le runtime RMI pour communiquer avec le serveur.

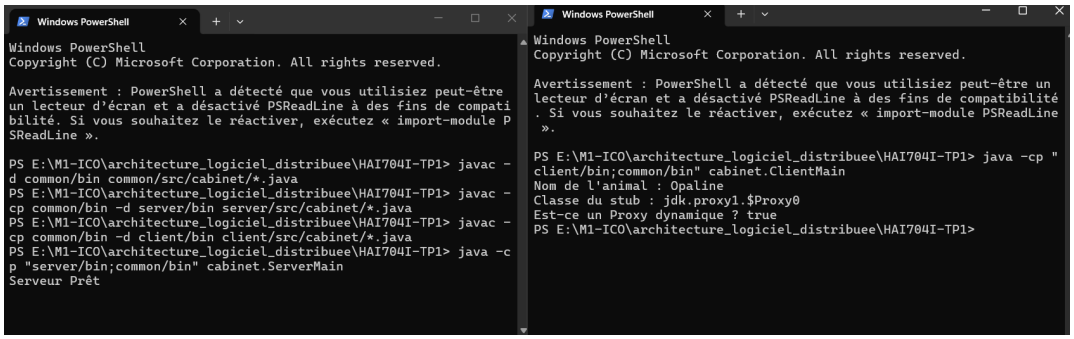


FIGURE 1 – Étape A1 : Récupération de l'animal et validation du Proxy dynamique.

4 Étape A2 : Type d'espèce transmis par valeur

L'espèce d'un animal est modélisée par une classe `Espece` (nom, espérance de vie) placée dans le module partagé `common` et implémentant `java.io.Serializable`.

Lorsqu'un client demande l'espèce d'un animal via `stub.getEspece()`, l'objet est transmis par valeur : il est sérialisé en un flux d'octets sur le réseau, puis reconstruit dans la JVM du client.

Pour vérifier ce comportement, le client modifie localement le nom de l'espèce reçue avec le setter `setNom("Chat")`. Une nouvelle interrogation du serveur confirme que l'espèce côté serveur est restée inchangée, valant toujours `"Chien"`.

De plus, l'affichage de `System.identityHashCode` donne des valeurs différentes entre le client et le serveur. En Java, cette méthode renvoie un code de hachage directement basé sur l'adresse mémoire de l'instance dans la machine virtuelle. Obtenir deux valeurs différentes prouve concrètement que le client manipule une copie autonome créée dans sa propre mémoire, et non l'objet présent sur le serveur.

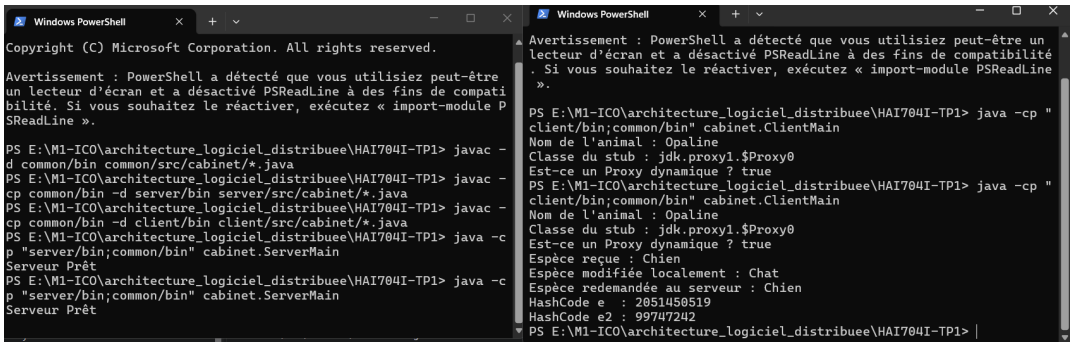


FIGURE 2 – Étape A2 : Modification locale de l'espèce sans effet sur le serveur.

5 Étape A3 : Dossier de suivi médical partagé

Chaque animal possède un dossier de suivi médical contenant son état de santé. Pour cette entité, le choix de conception est d'en faire un **objet distant** (passage par référence) et non un objet sérialisable.

En effet, la sérialisation produit une simple copie locale chez le client et ne permet pas d'avoir un état centralisé. Avec un objet distant, le dossier reste unique sur le serveur : plusieurs vétérinaires peuvent ainsi consulter et modifier le même dossier en temps réel.

L'interface `DossierSuivi` hérite donc de `Remote` et son implémentation `DossierSuiviImpl` réside sur le serveur. Lorsqu'un client invoque `dossier.setEtat("En convalescence")`, l'appel

transite via RMI et modifie l'état directement sur la JVM du serveur. Un second client relisant le dossier constate immédiatement la mise à jour.

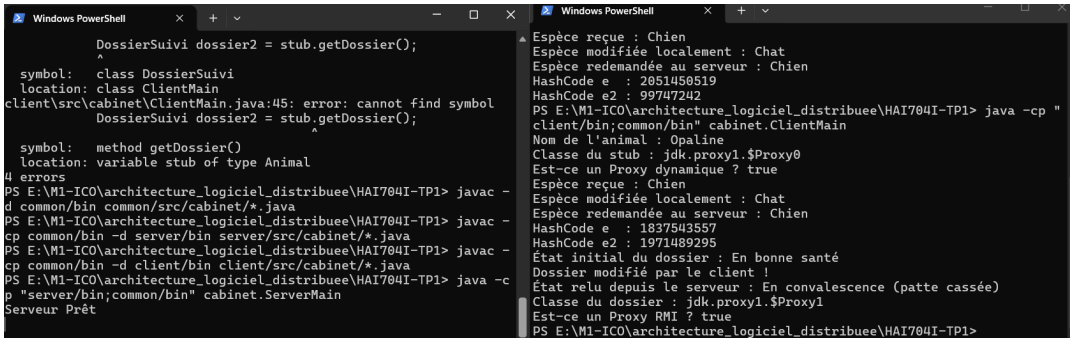


FIGURE 3 – Étape A3 : Modification et partage du dossier de suivi médical.

6 Étape A4 : Le cabinet vétérinaire (Point d'entrée unique)

Publier chaque animal et chaque dossier individuellement dans l'annuaire RMI n'est pas viable à grande échelle. L'interface distante `CabinetVeterinaire` devient donc l'unique objet publié dans le registre RMI.

Les contrats `Animal` et `DossierSuivi` restent dans le module partagé `common` pour définir les opérations accessibles au client, mais ne sont plus enregistrés dans l'annuaire. Le client contacte le registre une seule fois pour obtenir le stub du cabinet, puis navigue vers les patients et leurs dossiers médicaux à travers les méthodes distantes :

— `getPatients()` : renvoie la liste des stubs des animaux gérés par le cabinet.
— `rechercherPatient(String nom)` : recherche un animal par son nom côté serveur et renvoie sa référence distante (ou `null` si non trouvé).

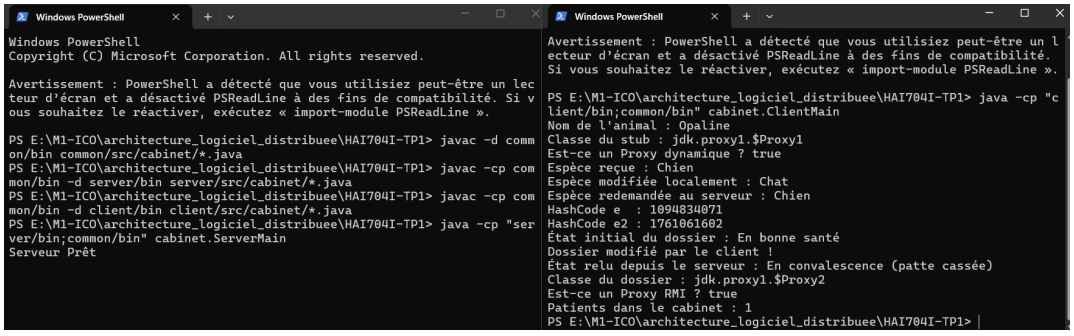


FIGURE 4 – Étape A4 : Point d'accès unique au cabinet et consultation des patients.

7 Étape A5 : Création d'un patient côté serveur

Un vétérinaire doit pouvoir enregistrer un nouvel animal depuis son poste. La méthode `ajouterPatient(nom, maitre, race, espece)` est donc ajoutée au contrat du cabinet.

En créant le patient sur le serveur, contrairement à une création locale, on garantit qu'il est immédiatement visible par l'ensemble des vétérinaires connectés et qu'il reste conservé même après la déconnexion du client.

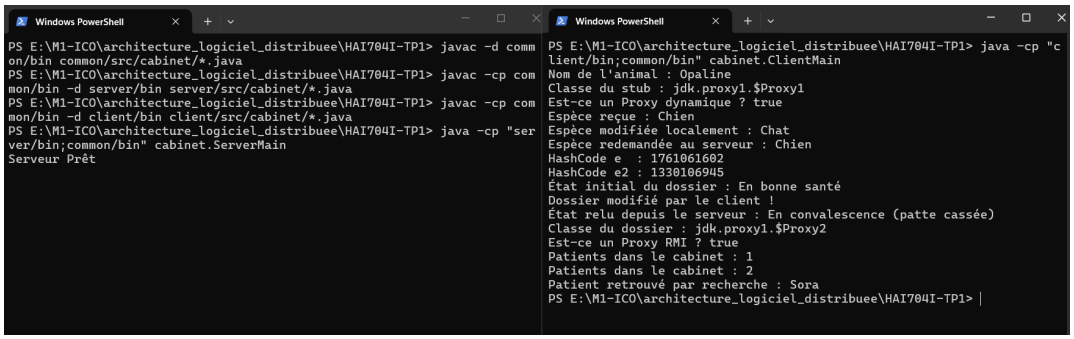


FIGURE 5 – Étape A5 : Ajout d'un patient et validation immédiate par recherche distante.

8 Étape A6 : Système d'alertes par callbacks (Abonnement)

Pour avertir automatiquement les vétérinaires lorsqu'un seuil de patients est franchi, on met en place le patron Observateur (callbacks) à travers le réseau.

Un contrat distant `CabinetObserver` est défini dans `common` avec la méthode `notifierAlerte(String msg)`. Côté client, la classe `ClientObserverImpl` implémente ce contrat et est exportée comme objet distant via `UnicastRemoteObject`. Lors de son démarrage, le client transmet son propre stub au serveur via `cabinet.sAbonner(stub)`.

À chaque ajout de patient, le serveur vérifie les seuils (configuré à 3 pour les tests) et notifie chaque client abonné. Pour assurer la robustesse du serveur, si un client est coupé brutalement, l'appel distant échoue en levant une `RemoteException`. Le serveur intercepte cette exception et retire automatiquement le client inaccessible de sa liste d'abonnés.

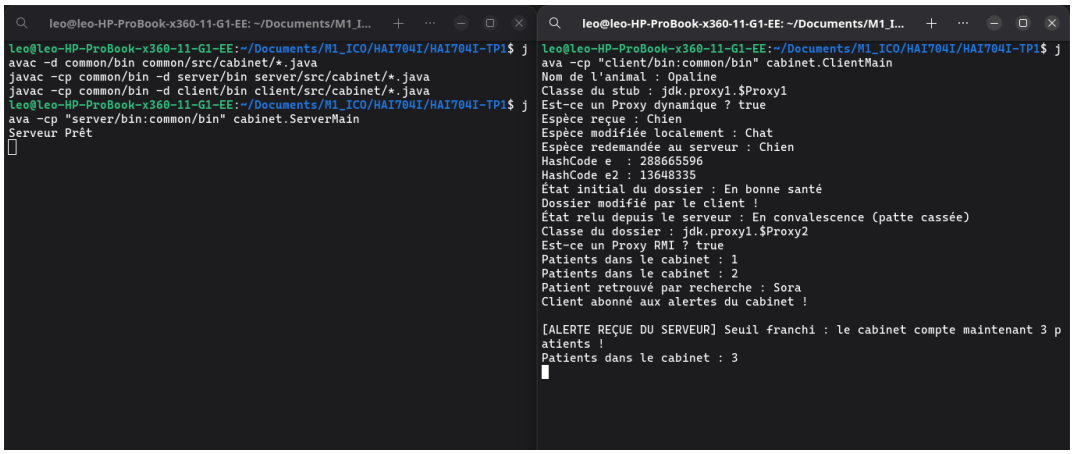


FIGURE 6 – Étape A6 : Réception instantanée d'une notification d'alerte sur le client.

9 Étape A7 : Ce qui traverse le réseau (Expérience de sérialisation)

Cette étape met en évidence ce qui transite réellement lors d'une transmission par valeur en RMI.

Pour cela, une sous-classe `Chien extends Espece` est volontairement créée uniquement côté client, sans que le serveur n'en ait connaissance. Lorsque le client tente d'enregistrer un patient avec cette instance, l'opération échoue immédiatement côté serveur.

L'extrait de trace ci-dessous (tronqué car le message d'erreur d'origine est très long) montre la cause de l'échec :

```
java.rmi.ServerException: RemoteException occurred in server thread;
nested exception is: java.rmi.UnmarshalException: error unmarshalling arguments;
nested exception is: java.lang.ClassNotFoundException: cabinet.Chien
```

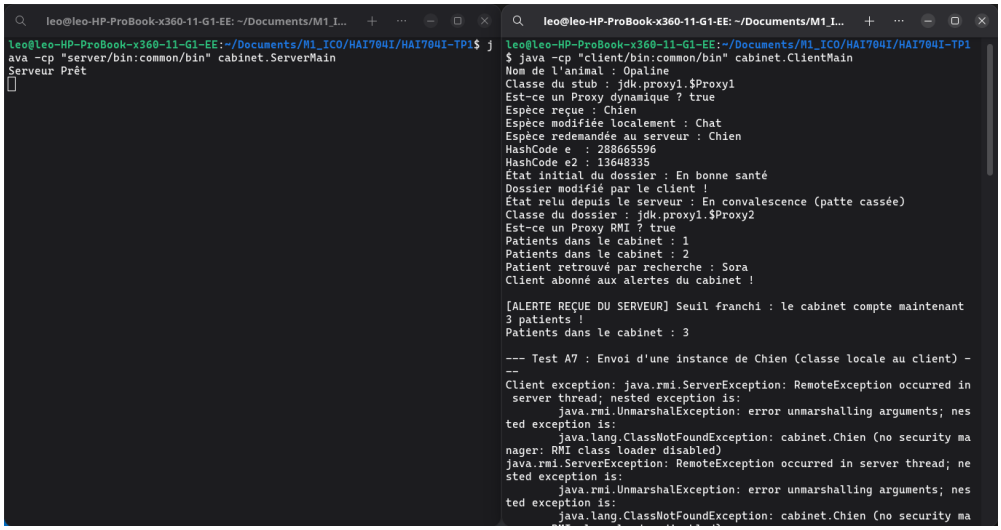


FIGURE 7 – Étape A7 : Erreur `ClassNotFoundException` dans la console cliente.

En effet, la sérialisation réseau transmet les valeurs des données et le nom de la classe, mais pas son code compilé. Le serveur doit donc obligatoirement posséder cette classe dans son propre classpath pour pouvoir reconstruire l'objet. Ne trouvant pas la classe `Chien` chez lui, le chargement échoue immédiatement.

Pour résoudre le problème, la classe `Chien` a simplement été déplacée dans le module partagé `common`. Une fois les modules recompilés, le serveur dispose du bytecode dans son classpath et l'ajout du patient s'exécute avec succès.

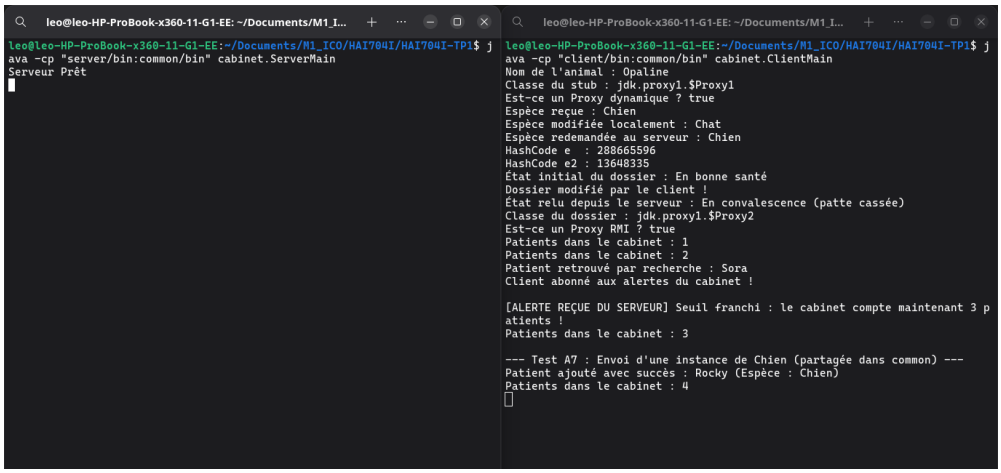


FIGURE 8 – Étape A7 : Validation de l'ajout après partage de la classe dans le module `common`.

10 Étape A8 : Application CLI interactive et architecture client

Pour proposer une application autonome utilisable sans recompiler le code, le client est structuré en trois responsabilités bien séparées :

1. `ClientMain` : point d'entrée, analyse des arguments (hôte et port), lookup du cabinet et démarrage de l'interface.

2. `CabinetService` : couche métier client qui encapsule tous les appels distants RMI, gère l'état de l'observateur et traite les erreurs réseau.
3. `ConsoleUI` : interface console interactive (menus, saisies utilisateur avec `Scanner`). **Cette classe ne contient aucun import `java.rmi.*`**, garantissant une vue totalement isolée du réseau.

Si la connexion et l'utilisation du client se déroulent sans difficulté, la déconnexion nécessite une attention particulière. En effet, comme le client exporte un observateur pour recevoir les alertes, le runtime RMI maintient des threads d'écoute actifs qui empêchent la machine virtuelle de s'arrêter. Lors du choix de quitter (option 0), le client invoque donc explicitement `unexportObject` sur son observateur pour libérer le port d'écoute et terminer proprement le processus.

De plus, un argument en ligne de commande `-no-test` a été ajouté afin de passer les tests automatiques (étapes A1 à A7) et d'accéder directement au menu interactif.



FIGURE 9 – Étape A8 : Menu interactif de la CLI cliente.

11 Partie C (Bonus) : Interface graphique Swing

Concernant l'interface graphique, j'ai fait le choix de rester en Java standard plutôt que de multiplier les langages ou les frameworks externes. J'ai donc choisi de développer une interface native en Java Swing (`SwingUI`), ce qui simplifie grandement l'intégration et garantit une exécution sans dépendance supplémentaire.

Cette vue graphique se branche directement sur la classe existante `CabinetService`, réutilisant la même logique métier sans aucune duplication de code RMI. De plus, les alertes réseau arrivant sur un thread d'arrière-plan du runtime RMI, leur affichage visuel est confié au fil d'exécution graphique via `SwingUtilities.invokeLater`, évitant tout blocage ou gel de la fenêtre.

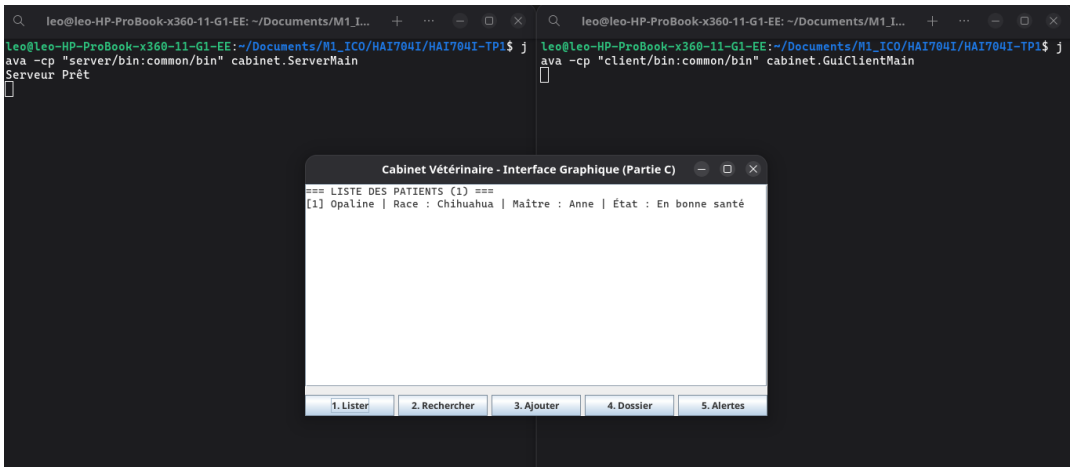


FIGURE 10 – Partie C : Interface graphique Swing connectée au service RMI.

12 Résultats et discussions

L'application finale valide les points de contrôle de A0 à A8 ainsi que le bonus C. Le système créé répond à la problématique initiale de gestion distribuée d'un cabinet vétérinaire en Java standard. Le registre RMI centralise un point d'accès unique, et les deux clients (console et graphique) interagissent en temps réel sur le même état partagé côté serveur.

Ayant réalisé le TP à la fois sous Linux et sous Windows, une première difficulté a concerné le séparateur de classpath (`:` sous Linux contre `;` sous Windows), ce qui m'a conduit à documenter clairement les deux syntaxes dans le README. Par ailleurs, la gestion des observateurs a demandé d'adapter le cycle de vie des processus : côté serveur, la capture ciblée de la `RemoteException` permet de purger la liste des abonnés lorsqu'un client est coupé brutalement, tandis que côté client, l'appel à `unexportObject` garantit l'extinction complète de la machine virtuelle.

Pour ce qui est des pistes d'amélioration, la première concerne la persistance des données sur disque. Les patients étant actuellement stockés uniquement en mémoire vive (RAM) sans fichier JSON ni base de données, un arrêt du serveur réinitialise toutes les fiches. La seconde concerne la sécurité et le contrôle d'accès : le serveur acceptant actuellement toutes les connexions entrantes, l'ajout d'un système d'authentification permettrait de réserver la modification des dossiers médicaux aux vétérinaires autorisés.

13 Usage des outils d'intelligence artificielle générative

Voici le périmètre d'utilisation de l'IA générative pour ce TP :

— J'ai utilisé l'IDE Antigravity développé par Google pour faire de la programmation agentique.

— En amont de l'écriture du code, j'ai sollicité l'IA pour m'aider à formaliser un cahier des charges afin de bien cerner les exigences et les attendus du sujet.

— Comme point de départ, je me suis basé sur le projet d'exemple `helloWorld` du TD 1, dont j'ai réalisé les premières adaptations moi-même pour construire l'étape A1.

— De l'étape A0 à A7, la structure et la logique métier ont été écrites en grande partie à la main, l'IA m'ayant assisté sur des éléments de syntaxe, les affichages de test et la correction d'erreurs.

— Pour l'étape A8 et la Partie C, le code étant assez verbeux, je me suis aidé de l'IA pour écrire un code suffisant pour être compris par moi-même, en veillant à ce qu'il utilise la syntaxe simple de Java standard.

— L'outil m'a conseillé sur plusieurs bonnes pratiques de travail, notamment l'isolation des fichiers compilés dans des répertoires `bin/` pour éviter de polluer le dépôt git.
— L'outil m'a également aidé à rédiger et structurer ce compte-rendu sous le format LaTeX, l'intégralité du document ayant ensuite été relue, vérifiée et adaptée par mes soins.

14 Références

1. Sujet du TP 1 et code d'exemple du TD 1 (projets de référence `hello-rmi`), Bachar Rima, Université de Montpellier.
2. Environnement de développement et assistant IA : *Google Antigravity IDE*, https://antigravity.google/product/antigravity-ide/.